



Chapter 13: Transactions

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Outline

- Transaction
- Schedules
 - Concurrent Executions
 - Serializability
 - Recoverability
- Transaction Isolation Levels



Transaction



Thinking

- n SQL Server中，Employee职工表包含1000000条记录，假设要执行以下语句

Update Employee Set salary = salary * 1.1

如果在执行到一半的时候，突然停电。那么重启后，Employee表会发生什么样的变化？



Transaction Concept

- A **transaction** is a *unit* of program execution that accesses and possibly updates various data items. (要么都做, 要么都不做)
- E.g., transaction to transfer \$50 from account A to account B:
 1. **read**(A)
 2. $A := A - 50$
 3. **write**(A)
 4. **read**(B)
 5. $B := B + 50$
 6. **write**(B)
- Two main issues to deal with:
 - Failures of various kinds, such as hardware failures and system crashes
 - Concurrent execution of multiple transactions



Example of Fund Transfer

- Transaction to transfer \$50 from account A to account B:
 1. **read**(A)
 2. $A := A - 50$
 3. **write**(A)
 4. **read**(B)
 5. $B := B + 50$
 6. **write**(B)
- **Atomicity requirement** — 原子性
 - If the transaction fails after step 3 and before step 6, money will be “lost” leading to an inconsistent database state
 - Failure could be due to software or hardware
 - The system should ensure that updates of a partially executed transaction are not reflected in the database
- **Durability requirement** — 持久性
 - once the user has been notified that the transaction has completed (i.e., the transfer of the \$50 has taken place), the updates to the database by the transaction must persist even if there are software or hardware failures.



Example of Fund Transfer (Cont.)

- **Consistency requirement** — 一致性
 - In above example:
 - The sum of A and B is unchanged by the execution of the transaction
- In general, consistency requirements include
 - Explicitly specified integrity constraints such as primary keys and foreign keys
 - Implicit integrity constraints
 - e.g., sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand
 - A transaction must see a consistent database.
 - During transaction execution the database may be temporarily inconsistent.
 - When the transaction completes successfully the database must be consistent
 - Erroneous transaction logic can lead to inconsistency



Example of Fund Transfer (Cont.)

■ Isolation requirement — 隔离性

- if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be).

T1

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

T2

read(A), read(B), print($A+B$)

- Isolation can be ensured trivially by running transactions **serially**
 - That is, one after the other.
- However, executing multiple transactions concurrently has significant benefits, as we will see later.



ACID Properties

A **transaction** is a unit of program execution that accesses and possibly updates various data items. To preserve the integrity of data the database system must ensure: 事务的四个特性(ACID)

- **Atomicity.** Either all operations of the transaction are properly reflected in the database or none are.
- **Consistency.** Execution of a transaction in isolation preserves the consistency of the database.
- **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
 - That is, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished.
- **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.



Thinking

- n 事务以Begin transaction语句开始，以Commit 或Rollback 语句结束
- n 下面的SQL程序包含几个事务？

Begin Transaction

Update account set money= money – 50 where no="A"

Update account set money= money + 50 where no="B"

Commit

- n 思考，如果执行完第一个update语句之后、没有执行第二个update语句之前，系统断电。那么在重启以后，两个帐户的金额会发生什么样的变化？



Thinking

n 下面的SQL程序包含几个事务？

Update account set money= money – 50 where no="A"

Update account set money= money + 50 where no="B"

n 思考，如果执行完第一个update语句之后、没有执行第二个update语句之前，系统断电，那么两个帐户的金额会发生什么样的变化？

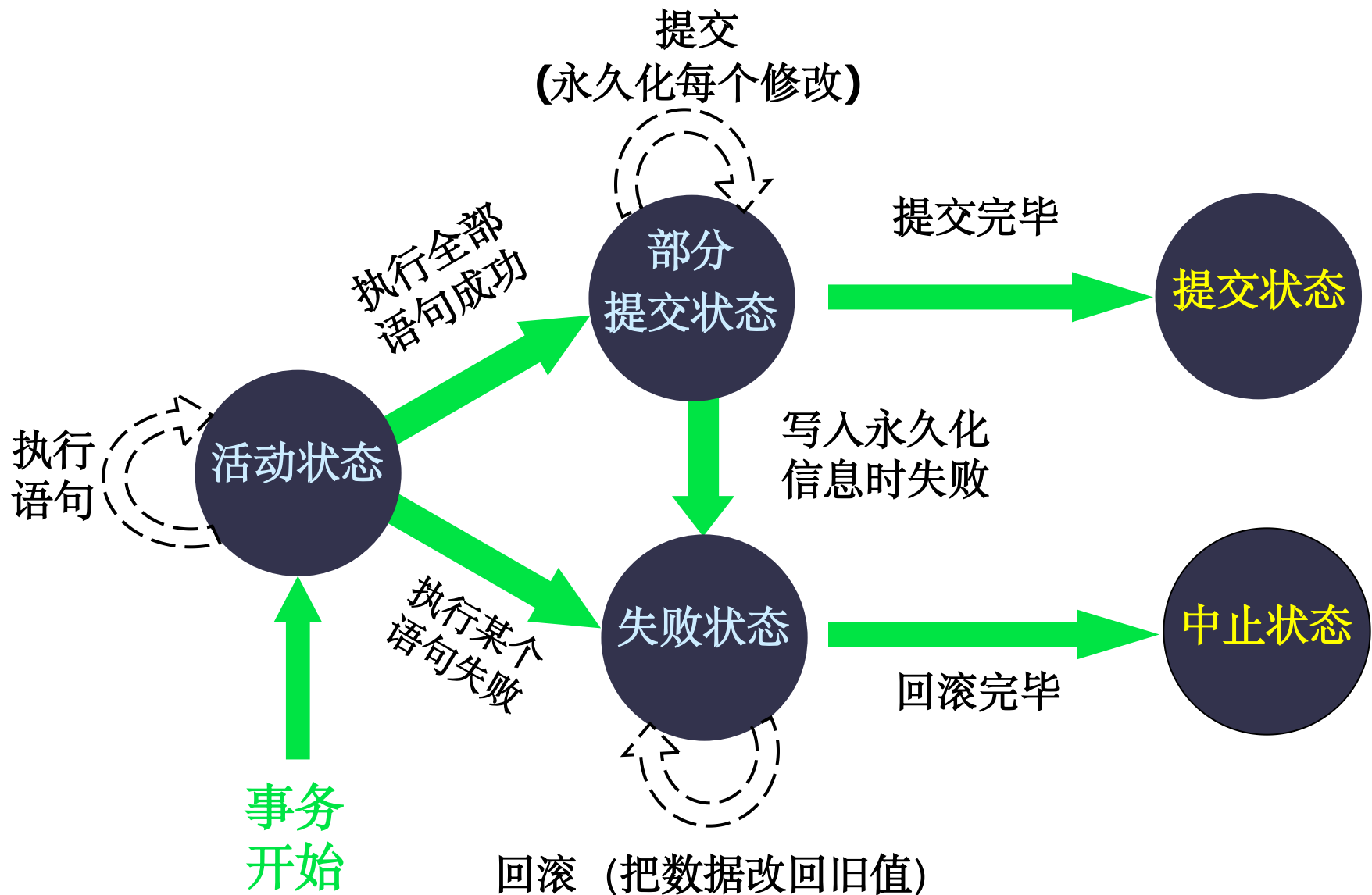


Transaction State

- **Active** – the initial state; the transaction stays in this state while it is executing
- **Partially committed** – after the final statement has been executed.
- **Failed** -- after the discovery that normal execution can no longer proceed.
- **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
 - Restart the transaction
 - Can be done only if no internal logical error
 - Kill the transaction
- **Committed** – after successful completion.



Transaction State





Transaction Schedules



Schedules

- **Schedule** – a sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed
 - A schedule for a set of transactions must consist of all instructions of those transactions
 - Must preserve the order in which the instructions appear in each individual transaction.
- A transaction that successfully completes its execution will have a commit instructions as the last statement
 - By default transaction assumed to execute commit instruction as its last step
- A transaction that fails to successfully complete its execution will have an abort instruction as the last statement



事务调度

n 事务调度

- | 指系统执行事务中指令的时间顺序
- | 一组多个事务的调度必须保证
 - ▶ 包含了所有事务的操作指令
 - ▶ 同一个事务中指令的相对顺序必须保持不变

n 两种类型

- | 串行调度
 - ▶ 执行完一个事务才开始执行下一个事务。因此从时间顺序上看，同一事务的指令紧挨在一起
- | 并发调度
 - ▶ 未执行完一个事务时，可转去执行另一个事务。因此从时间顺序上看，不同事务的指令彼此交叉



Concurrent Executions

- Multiple transactions are allowed to run concurrently in the system. Advantages are:
 - **Increased processor and disk utilization**, leading to better transaction *throughput*
 - E.g., one transaction can be using the CPU while another is reading from or writing to the disk
 - **Reduced average response time** for transactions: short transactions need not wait behind long ones.
- **Concurrency control schemes** – mechanisms to achieve isolation
 - That is, to control the interaction among the concurrent transactions in order to prevent them from destroying the consistency of the database
 - Will study in Chapter 14, after studying notion of correctness of concurrent executions.



Schedule 1 (串行)

- Let T_1 transfer \$50 from A to B , and T_2 transfer 10% of the balance from A to B .
- A **serial** schedule in which T_1 is followed by T_2 :

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit



Schedule 2 (串行)

- A serial schedule where T_2 is followed by T_1

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

- **Schedule 1 == Schedule 2 ? No**



Schedule 3 (并行)

- Let T_1 and T_2 be the transactions defined previously. The following schedule is not a serial schedule, but it is *equivalent* to Schedule 1

T_1	T_2
read (A) $A := A - 50$ write (A)	read (A) $temp := A * 0.1$ $A := A - temp$ write (A)
read (B) $B := B + 50$ write (B) commit	read (B) $B := B + temp$ write (B) commit

- In Schedules 1, 2 and 3, the sum $A + B$ is preserved.



Schedule 4

- The following concurrent schedule does not preserve the value of $(A + B)$.

T_1	T_2
read (A) $A := A - 50$	
	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B)
write (A) read (B) $B := B + 50$ write (B) commit	
	$B := B + temp$ write (B) commit



并发调度的问题

- n 并发调度的问题: 可能破坏数据一致性
 - | 即使单个事务的执行保证一致性, 不会破坏数据一致性; 多个这样的事务在并发调度时, 也可能破坏数据一致性
- n 产生问题的原因: 事务隔离性未得到保证
 - | 隔离性要求事务T1要么全部看到另一个事务T2对数据库的修改, 要么全部看不到
 - | 但在并发调度的情况下, 有时没有做到这一点
- n 解决问题的方法: 使并发调度可串行化
 - | 事务的串行调度显然不会破坏数据一致性
 - | 如果事务的并发调度是可串行化的, 即效果上等价于某个串行调度时, 也不会破坏数据一致性



三类数据不一致性

- n 错误的并发调度可能产生三种错误，又称为三类数据不一致性
 - | 丢失修改
 - | 不可重复读
 - | 读“脏”数据



第一类：丢失修改

n 丢失修改的产生

- | 并发调度两个事务T1、T2
- | T1与T2从数据库读入同一数据后分别修改。假设T1先提交，而T2后提交
- | 则T2提交的修改覆盖了T1提交的修改，导致T1的修改丢失



第一类：丢失修改

丢失修改的并发调度

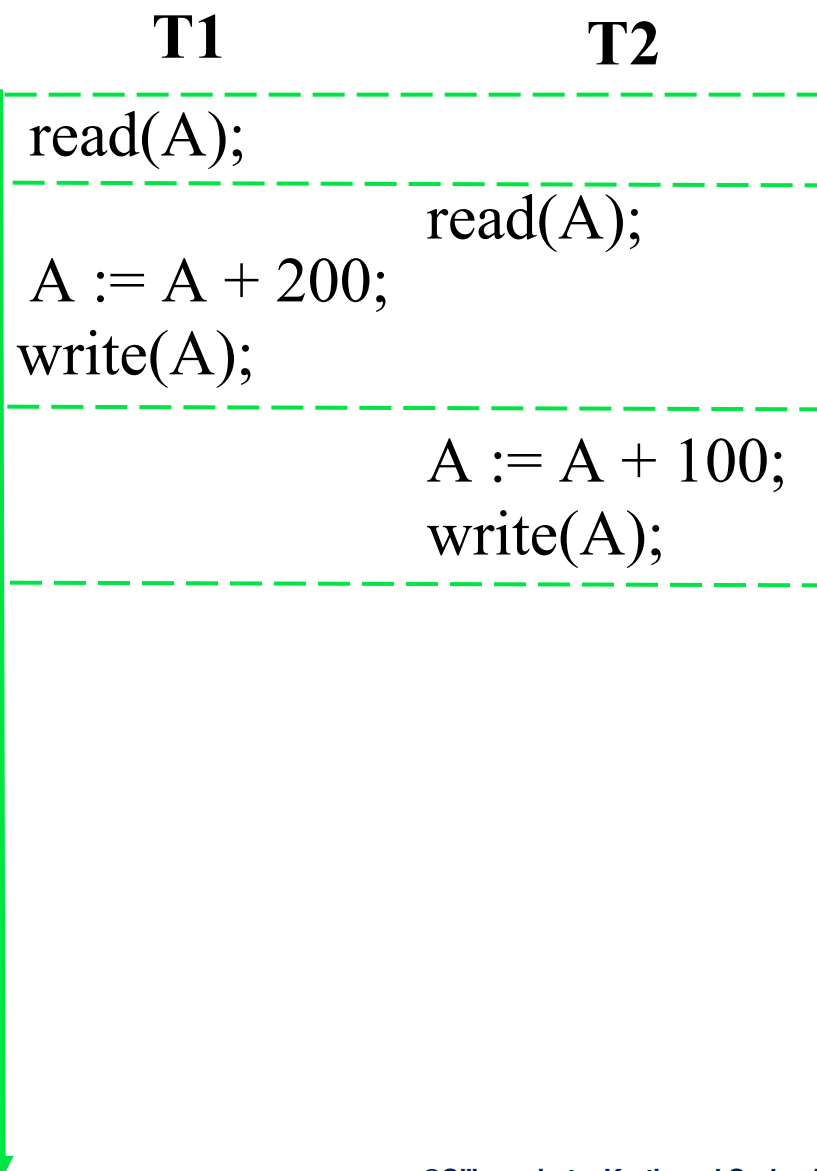
T1和T2读入同一数据A, T2提交的修改覆盖了T1提交的修改, 导致T1的修改丢失。

开始状态:

A=0 ¥

A=200 ¥

A=100 ¥





第二类：不可重复读

n 不可重复读的产生

- | 事务T1读取某些数据（记录）后，
- | 事务T2对这些数据（记录）做了某种修改操作
- | 当T1再次读取该数据（记录）时，得到的是与前一次不同的值



第二类：不可重复读

n 不可重复读又分为三种情况——

在事务T1两次读取之间，

- | 另一事务T2更新了记录。则当T1第二次读取时，得到与前一次不同的记录值
- | 另一事务T2删除了部分记录。则当T1第二次读取时，发现其中的某些记录神秘地消失了
- | 另一事务T2插入了一些记录。则当T1第二次按相同条件读取时，发现地神秘地多了一些记录

(后两种不可重复读有时也称为幻影现象)



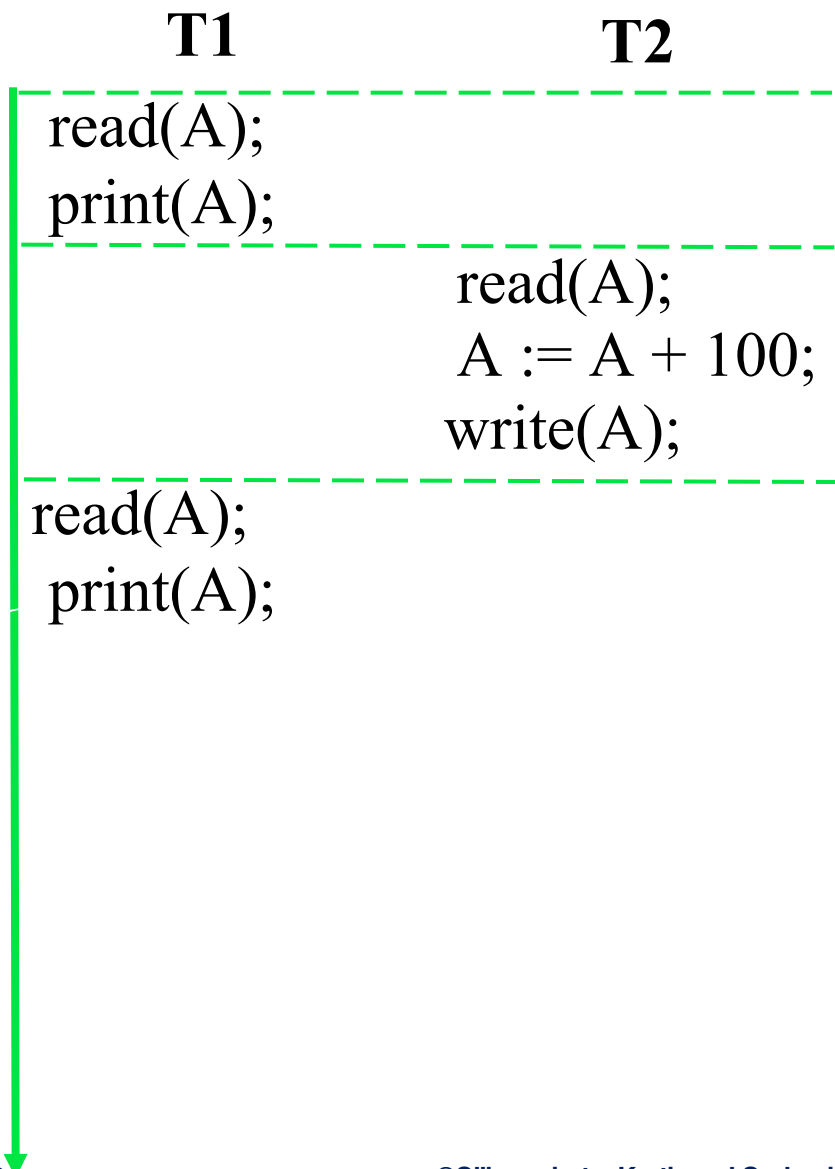
第二类：不可重复读

不可重复读的并发调度

事务T1读取某一数据A后，事务T2对其进行了更新操作，当T1再次读取该数据时，得到与前次不同的值。

打印A = ?

打印A = ?





第二类：不可重复读

n 幻影的例子

- | 事务T1: 查询年龄为23的学生，重复两次；事务T2: 插入一条学生记录

事务T1
Select * From S Where age=23;
Select * From S Where age=23;

串行调度T1、T2:
T1两次查询都
不包含学生“王”

事务T2
insert into S values(1, “王” , 23);

串行调度T2、T1:
T1两次查询都
包含学生“王”



第二类：不可重复读

| 产生“幻影”的并发调度

事务T1中，
两次执行相
同的查询有
不同结果。
前一次的查
询结果就是
“幻影”

两次查询的
结果是否相同？

T1

```
select *  
from S  
Where Age=23;
```

T2

```
insert into S  
values(1, “王”  
23);
```

```
select *  
from S  
Where Age=23;
```



第三类：读“脏”数据

n 读“脏”数据的产生

- | 事务T1修改某一数据，并写入数据库，但尚未结束(提交)
- | 事务T2读取同一数据，得到的是T1修改后的新值
- | 事务T1由于某种原因被撤消，则数据库中的数据恢复为修改前的旧值
- | 事务T2读到的数据就与数据库最终的数据不一致，是不正确的数据，又称为 “脏”数据 —— 其它事务修改后但又被撤销的数据。



第三类：读“脏”数据

n 读“脏”数据的例子

| 事务T1: 存款100后回滚；事务T2: 读取并打印存款额

起始状态:

$A=0$ ¥

事务T1
read(A); $A := A + 100$ write(A); RollBack;

事务T2
read(A); print(A);

串行调度T1、T2:

$A=0$ ¥; 打印的 **$A: 0$**

串行调度T1、T2:

$A=0$ ¥; 打印的 **$A: 0$**



第三类：读“脏”数据

读“脏”数据的并发调度

起始状态：
A = 0

T2读入**T1**修改过的数据**A**，尔后这个数据又被**T1**撤销了。致使**T2**读入的**A**是一个修改后被撤销的无效值，即“**脏**”数据。

A = 0

T1

```
read(A);  
A := A + 100;  
write(A);
```

T2

```
read(A);  
print(A);
```

Rollback;

打印**A=?**



第三类：读“脏”数据

n 广义的“脏”数据

- | 凡是另一事务修改过但是还没有提交的数据，对本事务来说都是“脏”的。
- | 读这种广义的“脏”数据是件冒风险的事情：可能会正确——如果最后对方事务提交了这种修改；也可能会出错——如果象前面的例子一样，对方事务最后把这种修改撤销了
- | 所以在严格要求正确性的场合，读“脏”数据是不允许的



Serializability (可串行化)

- **Basic Assumption** – Each transaction preserves database consistency.
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule.
- Thus, serial execution of a set of transactions preserves database consistency.
- Different forms of schedule equivalence give rise to the notions of:
 1. **Conflict serializability**



Simplified view of transactions

- We ignore operations other than **read** and **write** instructions
- We assume that transactions may perform arbitrary computations on data in local buffers in between reads and writes.
- Our simplified schedules consist of only **read** and **write** instructions.



Conflicting Instructions

- Instructions I_i and I_j of transactions T_i and T_j respectively, **conflict** if and only if there exists some item Q accessed by both I_i and I_j , and at least one of these instructions wrote Q .
 1. $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j don't conflict.
 2. $I_i = \text{read}(Q)$, $I_j = \text{write}(Q)$. They conflict.
 3. $I_i = \text{write}(Q)$, $I_j = \text{read}(Q)$. They conflict
 4. $I_i = \text{write}(Q)$, $I_j = \text{write}(Q)$. They conflict
- Intuitively, a conflict between I_i and I_j forces a (logical) temporal order between them.
- If I_i and I_j are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.



Conflict Serializability

- If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are **conflict equivalent**.
- *We say that a schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule*



Conflict Serializability (Cont.)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1 , by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

T_1	T_2
read (A) write (A)	read (A) write (A)
read (B) write (B)	
	read (B) write (B)

Schedule 3

T_1	T_2
read (A) write (A) read (B) write (B)	read (A) write (A) read (B) write (B)

Schedule 6



Conflict Serializability (Cont.)

- Example of a schedule that is not conflict serializable:

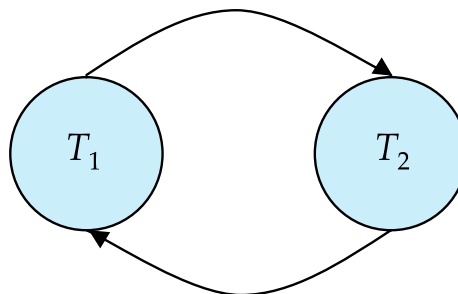
T_3	T_4
read (Q)	write (Q)
write (Q)	

- We are unable to swap instructions in the above schedule to obtain either the serial schedule $\langle T_3, T_4 \rangle$, or the serial schedule $\langle T_4, T_3 \rangle$.



Testing for Serializability

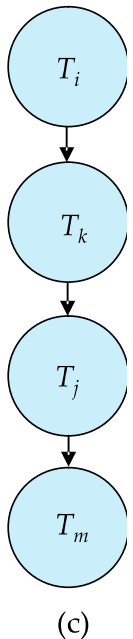
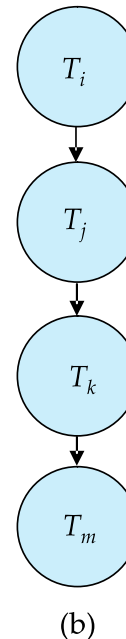
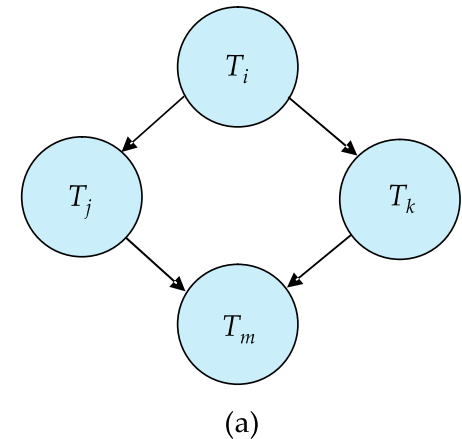
- Consider some schedule of a set of transactions $T_1, T_2, \dots, T_n$
- **Precedence graph** — a direct graph where the vertices are the transactions (names).
- We draw an arc from T_i to T_j if the two transaction conflict, and T_i accessed the data item on which the conflict arose earlier.
- We may label the arc by the item that was accessed.
- Example of a precedence graph





Test for Conflict Serializability

- A schedule is conflict serializable if and only if its precedence graph is acyclic.
- Cycle-detection algorithms exist which take order n^2 time, where n is the number of vertices in the graph.
 - (Better algorithms take order $n + e$ where e is the number of edges.)
- If precedence graph is acyclic, the serializability order can be obtained by a *topological sorting* of the graph.
 - This is a linear order consistent with the partial order of the graph.
 - For example, a serializability order for Schedule A would be
 $T_5 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_4$
 - Are there others?





Recoverable Schedules

Need to address the effect of transaction failures on concurrently running transactions.

- **Recoverable schedule** — if a transaction T_j reads a data item previously written by a transaction T_i , then the commit operation of T_i appears before the commit operation of T_j .
- The following schedule (Schedule 11) is not recoverable

T_8	T_9
read (A)	
write (A)	
	read (A)
	commit
read (B)	

- If T_8 should abort, T_9 would have read (and possibly shown to the user) an inconsistent database state. Hence, database must ensure that schedules are recoverable.



Cascading Rollbacks

- **Cascading rollback** – a single transaction failure leads to a series of transaction rollbacks. Consider the following schedule where none of the transactions has yet committed (so the schedule is recoverable)

T_{10}	T_{11}	T_{12}
read (A) read (B) write (A)	read (A) write (A)	read (A)
abort		

If T_{10} fails, T_{11} and T_{12} must also be rolled back.

- Can lead to the undoing of a significant amount of work



Cascadeless Schedules

- **Cascadeless schedules** — cascading rollbacks cannot occur;
 - For each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the commit operation of T_i appears before the read operation of T_j .
- Every Cascadeless schedule is also recoverable
- It is desirable to restrict the schedules to those that are cascadeless



Transaction Isolation Levels



Concurrency Control

- A database must provide a mechanism that will ensure that all possible schedules are
 - either conflict or view serializable, and
 - are recoverable and preferably cascadeless
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency
 - Are serial schedules recoverable/cascadeless?
- Testing a schedule for serializability *after* it has executed is a little too late!
- **Goal** – to develop concurrency control protocols that will assure serializability.



Concurrency Control (Cont.)

- Schedules must be conflict or view serializable, and recoverable, for the sake of database consistency, and preferably cascadeless.
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency.
- Concurrency-control schemes tradeoff between the amount of concurrency they allow and the amount of overhead that they incur.
- Some schemes allow only conflict-serializable schedules to be generated, while others allow view-serializable schedules that are not conflict-serializable.



Concurrency Control vs. Serializability Tests

- Concurrency-control protocols allow concurrent schedules, but ensure that the schedules are conflict/view serializable, and are recoverable and cascadeless .
- Concurrency control protocols (generally) do not examine the precedence graph as it is being created
 - Instead a protocol imposes a discipline that avoids non-serializable schedules.
- Different concurrency control protocols provide different tradeoffs between the amount of concurrency they allow and the amount of overhead that they incur.
- Tests for serializability help us understand why a concurrency control protocol is correct.



Weak Levels of Consistency

- Some applications are willing to live with weak levels of consistency, allowing schedules that are not serializable
 - E.g., a read-only transaction that wants to get an approximate total balance of all accounts
 - E.g., database statistics computed for query optimization can be approximate (why?)
 - Such transactions need not be serializable with respect to other transactions
- Tradeoff accuracy for performance



Levels of Consistency in SQL-92

- **Serializable** — default
- **Repeatable read** — only committed records to be read.
 - Repeated reads of same record must return same value.
 - However, a transaction may not be serializable – it may find some records inserted by a transaction but not find others.
- **Read committed** — only committed records can be read.
 - Successive reads of record may return different (but committed) values.
- **Read uncommitted** — even uncommitted records may be read.



Levels of Consistency

- Lower degrees of consistency useful for gathering approximate information about the database
- Warning: some database systems do not ensure serializable schedules by default
- E.g., Oracle (and PostgreSQL prior to version 9) by default support a level of consistency called snapshot isolation (not part of the SQL standard)



Transaction Definition in SQL

- In SQL, a transaction begins implicitly.
- A transaction in SQL ends by:
 - **Commit work** commits current transaction and begins a new one.
 - **Rollback work** causes current transaction to abort.
- In almost all database systems, by default, every SQL statement also commits implicitly if it executes successfully
 - Implicit commit can be turned off by a database directive
 - E.g., in JDBC -- `connection.setAutoCommit(false);`
- Isolation level can be set at database level
- Isolation level can be changed at start of transaction
 - E.g. In SQL **set transaction isolation level serializable**
 - E.g. in JDBC --
`connection.setTransactionIsolation`
(
`Connection.TRANSACTION_SERIALIZABLE`)



Implementation of Isolation Levels

- Locking
 - Lock on whole database vs lock on items
 - How long to hold lock?
 - Shared vs exclusive locks
- Timestamps
 - Transaction timestamp assigned e.g. when a transaction begins
 - Data items store two timestamps
 - Read timestamp
 - Write timestamp
 - Timestamps are used to detect out of order accesses
- Multiple versions of each data item
 - Allow transactions to read from a “snapshot” of the database



Transactions as SQL Statements

- E.g., Transaction 1:
select *ID, name* **from** *instructor* **where** *salary* > 90000
- E.g., Transaction 2:
insert into *instructor* **values** ('11111', 'James', 'Marketing', 100000)
- Suppose
 - T1 starts, finds tuples *salary* > 90000 using index and locks them
 - And then T2 executes.
 - Do T1 and T2 conflict? Does tuple level locking detect the conflict?
 - Instance of the **phantom phenomenon**
- Also consider T3 below, with Wu's salary = 90000
update *instructor*
set *salary* = *salary* * 1.1
where *name* = 'Wu'
- Key idea: Detect “**predicate**” conflicts, and use some form of “**predicate locking**”



End of Chapter 13